# A Computer-Vision-Based Bikeability Score

Bünyamin Aydemir, Gonzalo Guerrero Salazar, and Hendrick Fischer

***Abstract*—**

***Index Terms*—Bikeability, computer vision, zero-shot classification, vision-language models, CLIP, surface classification, object detection, GPX.**

## I. Introduction

Computer vision is one of the technologies that has firmly established itself in recent decades. The increase in computing capacity, the abundance of data, and the improvement of algorithms have propelled it—just as they have other areas of artificial intelligence—to the point where the technology is becoming a standard tool. We now view it as the de-facto solution to problems that previously required entirely different methods.

Take ADAS, for example, which are now expected in passenger vehicles rather than viewed as luxury equipment. However, these systems have largely not made their way to smaller vehicles like motorcycles or bicycles, the latter of which provide transportation and recreation to millions of people. Regular bicycle commuting inspired this project to implement vision systems not only to assess the safety, but also the aesthetic—and therefore mood-improving—aspects of bicycle routes. We call this metric the "bikeability score", based on the premise that while all roads are technically drivable, they do not all provide the same riding experience.

The score singles out three distinct factors that define this experience:

- **The road itself (Surface):** Determining the type of surface the user is riding on to assess if the vehicle is adequate and the ride will be physically comfortable.
- **The obstacles along the route (Traffic Dynamics):** Evaluating whether traffic is dominated by other bicycles or cars, and identifying major crossings to determine if the ride can remain largely continuous.
- **The surroundings (Environment):** Recognizing whether the route is dominated by cluttered cityscapes or greenery, and detecting natural features like forests and rivers to assess if the route is enjoyable rather than just a functional commute.

These three factors are defined as our target sub-functions, as each contributes differently to the riding experience and operates under distinct dynamics. Consequently, each task required a dedicated approach, making it essential to select models specifically suited to each sub-function. Similarly, dataset annotation and labeling were tailored to the specific demands of each individual task. The following sections explain the methodology for selecting, analyzing, and adapting these individual models. The final section details how the overall score is calculated from the combined outputs of each pipeline.

Manuscript created July 2026. The authors are students at the Baden-Württemberg Cooperative State University (DHBW) within the Computer Vision course.

## II. Research Approach and Methodology

The system is implemented as a multi-stream pipeline. Video data, collected using a handlebar-mounted DJI camera, is sampled at a frequency of one frame every five seconds. This dataset is passed in batches of eight to the individual detector functions. The resulting feature arrays are then processed by the primary scoring function, with each frame's evaluated score appended to an output file representing the main product of this analysis. Downstream from this output, several interpretation methods are possible, including min/max analysis and score averaging. For this project, a visualization was also generated by integrating GPS data.

The methodology initially leveraged pre-trained and zero-shot models where possible to minimize manual labeling effort and allow the taxonomy to evolve. However, as demonstrated in subsequent sections, this approach encountered specific limitations. As a result, each detector was independently benchmarked before integration into the final pipeline.

## III. Objectives and Constraints

The primary objective is to demonstrate a functional pipeline capable of evaluating real-world data. Consequently, testing the framework on self-collected footage was a strict requirement. However, this imposed a key constraint: ground-truth labeling had to be generated internally, either manually or with semi-automated assistance. This limitation constrained both the volume of real-world ground truth data available and the scope of benchmarking that could be conducted on candidate models.

## IV. Road-Surface Classification

The ridden surface strongly affects cycling comfort and safety: a paved cycleway is effortless, whereas loose gravel or bare soil is strenuous and, on a road bike, hazardous. Ground detection assigns each sampled frame a single surface class and supplies the surface term of the score. Frames come from a handlebar-mounted camera at one frame every five seconds; as each is dominated by one surface, the task is single-label classification, not segmentation.

### A. Task Definition and Model Selection

Two model families can solve this task: a classifier *trained on our own labels*, or a *zero-shot* vision-language model (VLM). We avoid a self-trained classifier, which would need a large, balanced, manually labeled training set and re-training on every change of the class definitions. A zero-shot VLM of the CLIP family [1] instead classifies an image by its similarity to natural-language class descriptions, so the taxonomy is adapted by editing text alone.

### B. Surface Classes and Prompts

We distinguish four cycling-relevant classes: **Cycleway** (a dedicated paved bike path), **Road** (a paved road for motor traffic), **Gravel** (loose crushed stone), and **Unpaved** (soil, mud, or field tracks). Each class is described by several English prompts whose normalized embeddings are averaged (*prompt ensembling*). As Cycleway and Road share the same smooth asphalt, the prompts encode the *discriminating context* (a narrow separated bicycle lane versus a wide multi-lane road) to separate the two embeddings. A frame is assigned to the class of highest cosine similarity.

### C. Evaluation Protocol

We compare nine open CLIP-family models of light to medium size through the unified `open_clip` API [2]: OpenAI CLIP, OpenCLIP (LAION-2B, DataComp-XL) [3], SigLIP [4], MobileCLIP [5], and EVA-CLIP [6]; both *accuracy* and *latency* matter for on-bike use. For the quantitative evaluation we manually labeled more than 1,700 frames from self-recorded rides, excluding frames without a clear surface. As the classes are imbalanced, we report accuracy, balanced accuracy and macro-F1 with 95% bootstrap confidence intervals and McNemar tests; latency is measured at batch size 1 with warm-up runs (ms/frame and FPS).

### D. Results

Table I lists the outcome, sorted by macro-F1. The best model, **OpenCLIP ViT-B/32 (LAION-2B)**, reaches macro-F1 0.58 at only 115 ms/frame, the best accuracy-latency trade-off. Two points stand out. First, the highest plain *accuracy* (0.78, CLIP ViT-B/16) does *not* give the highest macro-F1: accuracy rewards the majority classes, whereas macro-F1 weights all four surfaces equally and is the more faithful measure on imbalanced data. Second, the 95% bootstrap accuracy intervals of the leading models overlap, so their differences are *not* statistically significant; only CLIP ViT-L/14 (428 M) falls clearly below them; the largest model is both slowest and weakest (0.42 macro-F1), confirming that capacity alone does not help.

Table I
Zero-shot surface-classification benchmark on the self-labeled test set (>1,700 frames), sorted by macro-F1. BAcc: balanced accuracy; latency at batch size 1.

| Model | P[M] | Acc | BAcc | mF1 | ms | FPS |
|---|---|---|---|---|---|---|
| OpenCLIP B/32 (LAION-2B) | 151 | 0.75 | **0.72** | **0.58** | 115 | 8.7 |
| SigLIP B/16 (WebLI) | 203 | 0.75 | 0.62 | 0.57 | 259 | 3.9 |
| CLIP B/16 (OpenAI) | 150 | **0.78** | 0.62 | 0.57 | 261 | 3.8 |
| MobileCLIP-S2 | 99 | 0.73 | 0.70 | 0.56 | 304 | 3.3 |
| CLIP B/32 (OpenAI) | 151 | 0.70 | 0.60 | 0.54 | 93 | 10.7 |
| EVA02-B/16 | 150 | 0.76 | 0.62 | 0.54 | 304 | 3.3 |
| OpenCLIP B/32 (DataComp-XL) | 151 | 0.75 | 0.65 | 0.52 | 125 | 8.0 |
| CLIP L/14 (OpenAI) | 428 | 0.64 | 0.50 | 0.42 | 929 | 1.1 |

Broken down per class, the best model scores F1 0.80 (Cycleway) and 0.73 (Road) but only 0.45 (Gravel) and 0.33 (Unpaved); as macro-F1 averages the four equally, the two strong classes are pulled down by the two weak ones (Fig. 1). Crucially, the weakness is one of *precision*, not recall: Unpaved is still recalled well (0.77), yet its precision collapses to 0.21 because, with only 13 true frames, a few false positives bleeding in from the majority classes suffice to ruin it; Gravel (51 frames) behaves the same way, while the residual Cycleway/Road confusion (both smooth asphalt) is comparatively minor. A macro-F1 of 0.58 is thus by design strict on such tiny classes; the accuracy (0.75) and balanced accuracy (0.72, the highest of all models) are more flattering yet still fair, and for a zero-shot classifier on strongly imbalanced data the result is solid. We therefore adopt OpenCLIP ViT-B/32 (LAION-2B) as the ground detector; as all models share the same pipeline, the differences reflect architecture and training data (latencies are hardware-dependent).

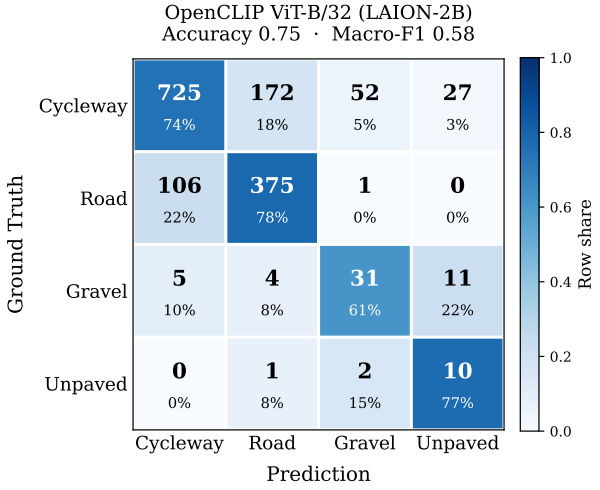



Figure 1. Row-normalized confusion matrix of the best model (OpenCLIP ViT-B/32, LAION-2B) on the labeled test set. Each cell shows the count and the row share; the diagonal is correct. Errors concentrate on the Cycleway/Road pair and the rare Gravel/Unpaved classes.

## V. Object Detection

For this evaluator, the methodology focuses specifically on counting objects. This design choice was made because objects move relative to the bike rider, as the rider is in continuous motion, so an object's size within the field of view changes on each frame and is not a stable factor for calculations.

This led to another design consideration regarding the scope of the "bikeability score" evaluator: the same route will yield different results depending on the moment the footage is captured. For this algorithm to be usable product, it must be capable of running in real time on a device mounted on the bicycle. This requirement applies only to the object detection.

Therefore, aditional selection criteria for this specific task are the model's inference time and its overall size, which are related. Larger models can also extract more complex image features, creating a trade-off between inference time and accuracy, requiring to find an optimal balance for our model.

The fact that this module only requires object recognition with high efficiency, led directly to the YOLO family of models as best candidates for the task. However, it remained necessary to benchmark different generations and sizes of these models to determine the best option for our application case.

### A. Relevant Objects

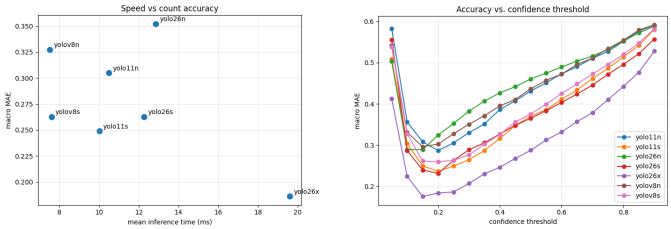
For the calculation of the score, the methodology restricts the categories of relevant objects. Rather than defining new categories from scratch, the project utilizes widely accepted object set categorizations. Because the system employs a pre-trained YOLO model, it is more efficient to leverage its baseline capabilities before resorting to custom retraining. Consequently, the COCO [7] dataset convention was selected as the reference, which consists of 80 object categories, not all of which relevant to street traffic. The relevant classes were grouped into three subsets: **motor vehicles**, **other road users**, and **traffic infrastructure**, from which representative classes were selected: **car**, **bicycle**, and **traffic light**.

The scoring heuristic designates cars as primary negative factor due to their physical size and potential speed. In contrast, the presence of bicycles acts as a mitigating buffer, offsetting penalties incurred by cars or traffic lights. Because the overall score is bounded, bicycles cannot independently increase the score beyond the baseline; they can only reduce existing penalties. Consequently, an empty road yields the maximum possible score.

### B. YOLO Benchmark

Initial benchmarking helps isolate the strongest candidate models. Trial runs were conducted on NVIDIA T4 GPUs via Google Colab, which represent a standard hardware platform for model benchmarking [8]. The T4 operates within the NVIDIA ecosystem—utilizing the CUDA API, cuDNN libraries, and dedicated execution engines—which closely mirrors the software architecture of edge devices like the NVIDIA Jetson series.

The YOLO26 family, released in January 2026, directly succeeds the YOLO11 lineup in the Ultralytics lineage. YOLO26 introduces an architecture that eliminates the need for Non-Maximum Suppression (NMS), reducing latency and making it a promising candidate for real-time processing, though it remains less battle-tested than its predecessors.




(a) (b)

Figure 2. YOLO model comparison for the object-detection module. (a) Mean inference time vs. macro MAE across YOLO variants. (b) Macro MAE vs. confidence threshold per model.

Comparing models at equivalent scales (nano and small) reveals that model size—rather than architectural generation—is the primary driver of performance and computational cost. At the small scale of the benchmark, YOLO11s achieves a lower macro Mean Absolute Error (MAE) than YOLO26s alongside comparable or faster inference, maintaining this performance margin across most confidence thresholds (Fig. 2(a)).

Larger variants like YOLO26x offer higher overall accuracy, but double the inference time and exceed target edge hardware limits. Model scaling carries a steep cost: stepping from nano to medium increases parameters by more than eightfold ($\sim$3.2 M to $\sim$25.9 M) and compute nearly tenfold (8.7 to 78.9 GFLOPs). Switching architectures from YOLO11s to YOLO26s adds a negligible $\sim$1 % increase in parameters (9.4 M to 9.5 M) with only modest gains. This kind of analysis is habitual in edge deployment scenarios. [9]

Because YOLO26 only outperforms YOLO11 when scaling up size, YOLO11s remains the stronger candidate here given its maturity, speed, and computational efficiency. Notably, these findings reflect behavior on our labeled data; performance on other datasets may vary.

### C. Effect of Confidence Thresholds on Model Performance

Confidence thresholding plays a critical role in balancing false positives and false negatives within object predictions. Across all evaluated models, the MAE drops sharply as the threshold is raised from very low values (0.05–0.15). In this range, a large volume of incorrect detections exhibits low confidence scores. Raising the threshold eliminates the false positives while retaining most true positives, up to a point beyond from which it filters out valid detections.

The tested models reach their minimum MAE at different confidence thresholds (Fig. 2(b)). This variation indicates differences in calibration across model architectures, confirming that both model selection and threshold configuration must be tuned specifically for the target application. For our dataset, YOLO11s achieves its optimal performance at a confidence threshold of approximately 0.20.

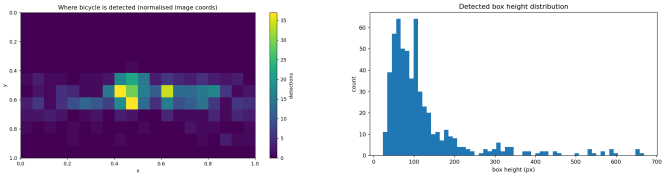



Figure 3. Distribution of bicycle detections across the dataset. (a) Spatial density of detections in normalised image coordinates. (b) Histogram of detected bounding box heights in pixels.

### *D. Dataset Distribution and Bounding Box Analysis*

An analysis of how the detected objects are distributed across the dataset to was carried out to better understand the data and evaluate the suitability of the chosen model.

Given the manual effort required to annotate object bounding boxes across the full dataset, this analysis was assisted by YOLO26x —the model that demonstrated the highest detection accuracy during preliminary evaluation, while exceeding the imposed computational constraints. An inference pass using YOLO26x provided bounding boxes across all frames. It was shown that cars represent the most frequent object class, while bicycles are significantly sparser and prove more challenging to detect.

Spatial localization of detections clusters tightly around the horizontal center of the image (roughly $x = 0.40$ to $0.65$) at mid-to-lower frame height ($y \approx 0.45$ to $0.65$), exhibiting two distinct density hotspots rather than a uniform distribution (Fig. 3(a)). As expected given the camera perspective, detections concentrate along the road surface, with density increasing near the vanishing point.

Object size correlates with distance from the camera. Farther objects appear smaller, potentially making them harder to detect. Bicycles represent the most difficult category to detect due to its relative scarcity, making the size distribution of detected bicycles particularly relevant. Across 1001 evaluated frames, 565 raw bicycle detections were recorded, with a median bounding box height of only 93 px (10th percentile $=$ 46 px, 90th percentile $=$ 228 px). This indicates that most bicycle detections occur while the vehicle is distant—located near the central vanishing point—with very few detections occurring at close range off to the side.

An initial hypothesis formulated during development suggested that the MAE for bicycles primarily stems from undercounting small, distant objects. At its baseline setting (confidence threshold $= 0.25$, minimum box height $=$ 0–30 px), the model predicted 154 bicycle instances, achieving an MAE of 0.127 and a bias of $-0.10$. Applying a minimum height filter of 70 px (effectively discarding small, distant detections near the center) degraded performance: the predicted count dropped to 125, and the MAE increased to 0.15.

These empirical results directly support the hypothesis: small, centered detections along the user's path represent critical instances that size-filtering or excessive downscaling would erroneously eliminate.

## VI. Environment Detection: Zero-Shot vs. Fine-Tuned Semantic Segmentation

Riding through different environments shapes the experience: green *vegetation* and *water* make a ride pleasant, too much *city* and traffic make it strenuous. Environment detection is a *multi-label whole-image* task (a frame may show several classes or none) and supplies the *largest* score weight ($w_e = 0.45$, Eq. (2)).

### A. Task and Model Selection

**Task:** find the best model to classify the environment on our own test frames, captured with a pod-mounted camera on the bike. **Comparison:** training-free *zero-shot CLIP* [1], [10] vs. a *fine-tuned SegFormer* [11].

### B. Model Comparison

**CLIP (zero-shot)** scores each frame against a per-class prompt pair in a shared embedding space, yielding *one present-probability per class* – no *where* or *how much* – thresholded into a label, and is tuned only via prompts and thresholds. **SegFormer (fine-tuned)** instead labels *every pixel* and reports each class's *area fraction* (pixels ÷ total), the *how much* the score consumes [12] (Eq. (4)); only it yields those fractions and a pixel-level mIoU, at the cost of training.

### C. Zero-Shot CLIP: Optimization

Two levers dominated (Table II): *per-class threshold calibration* on a leakage-free validation split (+0.095 macro-F1) and *prompt ensembling* [1] over 6–8 templates per class (+0.044); larger capacity (ViT-L/14) did *not* help. Adding the water rides in the final test (C7) then exposed CLIP's apparent water strength as a small-validation artifact.

Table II
CLIP zero-shot optimization steps: C1–C6 are pre-water; C7 adds the water rides. Final: ViT-B/32, prompt ensembling.

| # | Optimization step | Macro-F1 |
|---|---|---|
| C1 | Aligned prompt pairs, global threshold | 0.547 |
| C2 | Ride-disjoint val/test split | 0.632 |
| C3 | Per-class threshold calibration | 0.727 |
| C4 | Prompt ensembling, 6–8 templates | 0.771 |
| C5 | Larger backbone ViT-L/14 (rejected) | 0.756 |
| C6 | 3-class taxonomy (veg. merge) | **0.846** |
| C7 | Water-inclusive final test | 0.649 |

### D. Fine-Tuned SegFormer: Optimization

Tuning ran in two phases (Table III). *(i) Training* on Mapillary [13] pixel masks, by **mIoU**: the decisive lever was *data* – balanced oversampling plus 258 ADE20K water scenes [14] lifted water IoU 0→0.70, and an ADE20K base [15] with an "other" class raised it further. *(ii) Evaluation* on held-out own frames, by **macro-F1**, ending on the unseen water-inclusive test (0.704); ablation S10 values fine-tuning at +0.027.

Table III
SegFormer optimization in two phases: *training* on Mapillary pixel masks (mIoU, dev set), then *evaluation* on own frames (macro-F1).

| # | Optimization step | $\text{mIoU}_{\text{tr}}$ | $\text{F1}_{\text{eval}}$ |
|---|---|---|---|
| S1–2 | Cityscapes-B0, random Mapillary; LR tuning | 0.62 | – |
| S3–4 | Balanced + water aug + ADE water | 0.82 | – |
| S5 | Base swap → ADE20K (water prior) | **0.857** | – |
| S7 | First own eval + val thresholds | – | 0.636 |
| S8 | Explicit "other" background class | – | 0.707 |
| S9 | 3-class retrain (pre-water) | – | 0.874 |
| S10 | Fine-tuning ablation (no-FT) | – | 0.847 |
| S11 | Threshold grid, water-incl. final | – | **0.704** |

### E. Metrics

Two metrics on the **1,210**-frame own-test set (ride-disjoint from tuning; 983/40/307 veg/water/city positives). **Macro-F1**: per-class F1 (precision–recall harmonic mean) averaged equally over classes, so rare *water* counts fully. **Exact-match**: share of frames whose entire three-class label is correct at once.

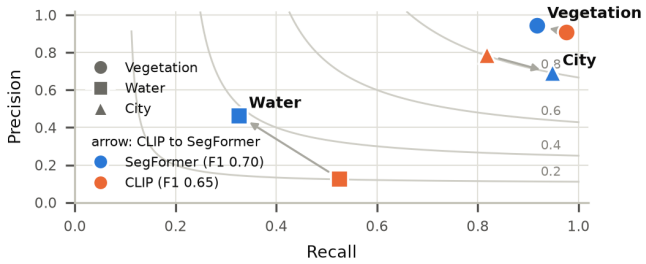



Figure 4. Precision–recall operating points with iso-F1 contours (own-test, 1,210 frames); arrows run CLIP→SegFormer. The isolated water pair (bottom-left) decides it: SegFormer trades recall for precision to reach a higher F1.

### F. Results and Decision

**Results.** SegFormer wins overall: macro-F1 0.704 vs. 0.649 and exact-match 0.755 vs. 0.691. The models tie on the common classes (vegetation 0.930/0.941, city 0.799/0.802), so the rare *water* class (3% of frames) decides it: SegFormer F1 0.382 vs. 0.206. Figure 4 shows two things: *(1)* the decision lives in the bottom-left – vegetation and city cluster on the same high-F1 band, only the water points are far apart; *(2)* SegFormer wins water by *precision, not recall*: CLIP reaches higher water recall (0.53 vs. 0.33) only by firing indiscriminately at precision 0.13, whereas SegFormer holds precision 0.46 for a higher F1.

**Decision.** We deploy the fine-tuned SegFormer: it wins the deciding class with trustworthy precision, ties CLIP elsewhere, is 40× smaller (3.7 M vs. 151 M), and feeds the score area fractions directly – which CLIP cannot. *Water stays the known limitation*: recall (∼67% of true-water frames missed) is capped by data scarcity, fixable only with more in-domain water.

## VII. Computation of the Bikeability Score

The bikeability score combines the three components (surface from ground-detection, environment from environment-detection, and objects from object-detection) into a single value in the interval $[0, 100]$. Every detector emits a fixed-length *vector*; each vector is first mapped to a *sub-score* in $[0, 1]$, and the three sub-scores are then combined by a weighted mean.

### A. Detector Outputs as Vectors

The three detectors deliberately use different but fixed output representations, each aligned element by element with a stored value vector:

- **Ground**: a one-hot vector $\mathbf{g} \in \{0, 1\}^4$ over the classes (Cycleway, Road, Gravel, Unpaved); exactly one entry is 1.
- **Environment**: a vector of area fractions $\mathbf{f} \in [0, 1]^3$ over (vegetation, water, city).
- **Object**: a vector of counts $\mathbf{n} \in \mathbb{Z}^3_{\geq 0}$ over (bicycle, car, traffic light).

### B. Overall Formula

For each evaluated frame, the score $A$ results from the weighted mean of the three sub-scores:

$$A = 100 \cdot \Big( w_g \, S_{\text{ground}} + w_e \, S_{\text{env}} + w_o \, S_{\text{object}} \Big), \qquad (1)$$

with the weights

$$w_g = 0.20, \quad w_e = 0.45, \quad w_o = 0.35. \qquad (2)$$

The weights are heuristic and follow two considerations. Environment and object exposure jointly capture how pleasant and how safe a stretch feels, the factors a cyclist perceives most directly, and therefore receive the larger share ($w_e + w_o = 0.80$). The surface receives the smallest weight ($w_g = 0.20$): on typical routes it varies little, and ground detection is also the least reliable component (macro-F1 0.58), so a smaller weight limits the influence of its errors. Since all sub-scores lie in $[0, 1]$ and the weights sum to 1, the score is bounded to $[0, 100]$. In practice the lower extreme 0 is never reached, as $S_{\text{ground}} \geq 0.1$ and $S_{\text{object}} = \exp(-\max(\rho, 0)) > 0$; the upper value 100, however, does occur whenever all three sub-scores equal 1 simultaneously, e.g. on a paved cycleway ($S_{\text{ground}} = 1$) through vegetation or along water ($S_{\text{env}} = 1$) with no motorised traffic ($S_{\text{object}} = 1$).

### C. Ground Sub-Score

The ground sub-score is the inner product of the one-hot vector $\mathbf{g}$ with the surface quality vector $\mathbf{q}^g = (1.0,\, 0.7,\, 0.3,\, 0.1)$ (Table IV):

$$S_{\text{ground}} = \langle \mathbf{g},\, \mathbf{q}^g \rangle = \sum_k g_k \, q^g_k. \qquad (3)$$

Because $\mathbf{g}$ is one-hot, this simply selects the quality of the detected surface class.

### D. Environment Sub-Score

The environment sub-score is the fraction-weighted mean of the environment quality vector $\mathbf{q}^e = (1.0,\, 1.0,\, 0.4)$:

$$S_{\text{env}} = \begin{cases} \dfrac{\langle \mathbf{f},\, \mathbf{q}^e \rangle}{\sum_k f_k}, & \text{if } \sum_k f_k > 0, \\ 0.5, & \text{otherwise.} \end{cases} \qquad (4)$$

The normalization by $\sum_k f_k$ keeps the sub-score in $[0, 1]$ even when the detected fractions do not cover the whole frame; a neutral value of 0.5 is used if no environment class is detected.

### E. Object Sub-Score

The object sub-score maps the counts $\mathbf{n}$ through a *saturation function* using the penalty vector $\boldsymbol{\lambda} = (-0.2,\, 1.0,\, 0.3)$:

$$S_{\text{object}} = \exp\Big( -\max\big(\rho, 0\big)\Big), \qquad \rho = \langle \mathbf{n},\, \boldsymbol{\lambda} \rangle, \qquad (5)$$

where $\rho$ is evaluated independently per frame. A positive factor for cars ($+1.0$) and traffic lights ($+0.3$) makes $\rho$ positive and thus lowers the sub-score, whereas bicycles carry a *negative* factor ($-0.2$). The clamp $\max(\rho, 0)$ makes the score asymmetric: a frame is never rewarded *above* the neutral value 1.0, so bicycles do not inflate the score on their own; they only *offset* the penalty of nearby cars or traffic lights. This encodes the intuition that good cycling conditions are the default and only motorised traffic degrades them. The exponential form additionally caps the influence of outliers, so a single crowded frame cannot dominate the overall score.

Table IV
Class value vectors for the three sub-scores: surface quality $\mathbf{q}^g$, environment quality $\mathbf{q}^e$, and object weight $\boldsymbol{\lambda}$ (heuristic; a negative value acts as a bonus).

| Component | Class | Value |
|---|---|---|
| Ground $\mathbf{q}^g$ | Cycleway | 1.0 |
| | Road | 0.7 |
| | Gravel | 0.3 |
| | Unpaved | 0.1 |
| Environment $\mathbf{q}^e$ | Vegetation | 1.0 |
| | Water | 1.0 |
| | City | 0.4 |
| Object weight $\boldsymbol{\lambda}$ | Bicycle | $-0.2$ |
| | Car | $+1.0$ |
| | Traffic light | $+0.3$ |

### F. GPX Mapping and Visualization

Because the camera clock is unreliable, the frames are aligned to the GPS track by a single *manual sync point* (by default: recording start = GPX track start). Each frame is then matched to its nearest GPX track point in time (a nearest-neighbor join within a 3 s tolerance: large enough to bridge gaps in the roughly 1 Hz GPS log, yet below the 5 s frame spacing so that each frame maps to a distinct track point), which assigns a latitude/longitude to every score. The *average* bikeability score of the route is the mean $\bar{A} = \frac{1}{M} \sum_{i=1}^{M} A_i$ over all $M$ evaluated frames. Finally, the route is drawn as a sequence of short segments, each colored by the local score on a red–orange–yellow–green scale (Fig. 5). For the example

ride shown, the mean score is $\bar{A} \approx 85.7/100$: the track is predominantly green (good cycling conditions) with a few yellow/orange sections.

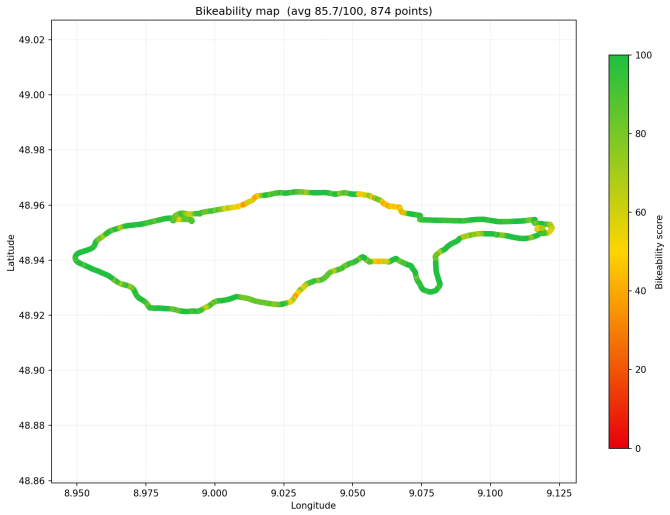



Figure 5. Bikeability score colored along the GPX route (red = low, green = high). Per-frame scores are matched to the nearest GPS track point via a manual sync point; the example ride has a mean score of $\bar{A} \approx 85.7/100$.

## VIII. Discussion

## IX. Conclusion

## References

[1] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proc. Int. Conf. Machine Learning (ICML)*, 2021, pp. 8748–8763.

[2] G. Ilharco, M. Wortsman, R. Wightman, C. Gordon, N. Carlini, R. Taori, A. Dave, V. Shankar, H. Namkoong, J. Miller, H. Hajishirzi, A. Farhadi, and L. Schmidt, "OpenCLIP," Zenodo, 2021, https://github.com/mlfoundations/open_clip.

[3] S. Y. Gadre, G. Ilharco, A. Fang *et al.*, "DataComp: In search of the next generation of multimodal datasets," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[4] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid loss for language image pre-training," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 11 975–11 986.

[5] P. K. A. Vasu, H. Pouransari, F. Faghri, R. Vemulapalli, and O. Tuzel, "MobileCLIP: Fast image-text models through multi-modal reinforced training," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2024.

[6] Q. Sun, Y. Fang, L. Wu, X. Wang, and Y. Cao, "EVA-CLIP: Improved training techniques for CLIP at scale," *arXiv preprint arXiv:2303.15389*, 2023.

[7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft COCO: Common objects in context," in *Computer Vision – ECCV 2014*, 2014, pp. 740–755.

[8] G. Jocher, J. Qiu, M. Liu, S. Lyu, F. C. Akyon, and M. E. Kalfaoglu, "Ultralytics YOLO26: Unified real-time end-to-end vision models," *arXiv preprint arXiv:2606.03748*, 2026.

[9] P. Martínez Otero, A. Tellaeche, and M. Hernández Melero, "Performance analysis of the YOLO object detection algorithm in embedded systems: Generated code vs. native implementation," *Computation*, vol. 14, no. 3, p. 67, Mar. 2026.

[10] W. Huang, J. Wang, and G. Cong, "Zero-shot urban function inference with street view images through prompting a pretrained vision-language model," *Int. J. Geographical Information Science (IJGIS)*, vol. 38, no. 7, pp. 1414–1442, 2024.

[11] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "SegFormer: Simple and efficient design for semantic segmentation with transformers," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 12 077–12 090.

[12] D. H. Lee, H. Y. Park, and J. Lee, "A review on recent deep learning-based semantic segmentation for urban greenness measurement," *Sensors*, vol. 24, no. 7, p. 2245, 2024.

[13] G. Neuhold, T. Ollmann, S. Rota Bulò, and P. Kontschieder, "The Mapillary Vistas dataset for semantic understanding of street scenes," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 5000–5009.

[14] B. Zhou, H. Zhao, X. Puig, T. Xiao, S. Fidler, A. Barriuso, and A. Torralba, "Semantic understanding of scenes through the ADE20K dataset," *Int. J. Computer Vision (IJCV)*, vol. 127, no. 3, pp. 302–321, 2019.

[15] T. Cornille and N. Rogge, "Fine-tune a semantic segmentation model with a custom dataset," Hugging Face Blog, 2022, https://huggingface.co/blog/fine-tune-segformer.